Phase 2 - Proposal and Code Implementation

Subject: Pattern Recognition

Topic: Vehicle Type Classification

Submitted By:

Tania Girish

M.Sc. Software Engineering

University of Europe for Applied Sciences

Supervisor: Raja Hasim Ali

Date: June 6 2026

Table of Contents

1. Background and Motivation

Vehicle type classification is an important task in traffic management and smart city systems. Automatically identifying vehicles such as cars, buses, trucks, motorcycles, and bicycles from images helps with traffic counting, parking management, and road monitoring. Traditional manual methods are slow and not scalable, so deep learning using Convolutional Neural Networks (CNNs) provides a better automated solution.

This project builds a CNN-based system to classify vehicle types from images using four different models: a Custom CNN, MobileNetV2, ResNet50, and EfficientNetB0. The system is designed to assist human operators rather than replace them, making it a decision-support tool for real-world traffic scenarios.

2. Problem Statement

Classifying vehicle types from images is challenging because different vehicles can look similar (e.g., a van and a truck), and images can have different lighting, angles, and backgrounds. This project addresses these challenges by comparing a simple custom CNN baseline with three transfer learning models and explaining model decisions using Grad-CAM visualizations. The goal is to find the most accurate and efficient model for vehicle type classification.

3. Dataset Used

The dataset used is the Vehicle Image Classification Dataset from Kaggle. It contains 5,600 high-resolution images across 7 vehicle classes with approximately 800 images per class.

Attribute	Details
Dataset Name	Vehicle Image Classification Dataset
Source	Kaggle
Total Images	5,600 images
Number of Classes	7 (Bicycle, Bus, Car, Motorcycle, Pickup Truck, Truck, Van)
Image Format	JPEG / PNG, RGB
License	CC BY-SA 4.0 (Academic use permitted)
Train / Val / Test Split	70% / 15% / 15% (stratified)

Link – https://www.kaggle.com/datasets/mohamedmaher5/vehicle-classification

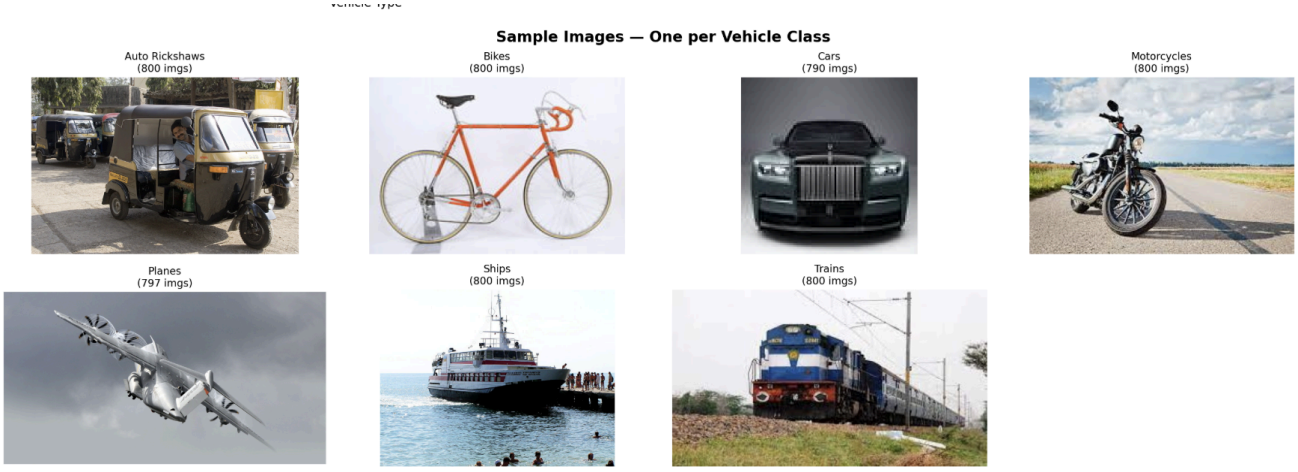

Fig 1. Sample images from each vehicle class in the dataset

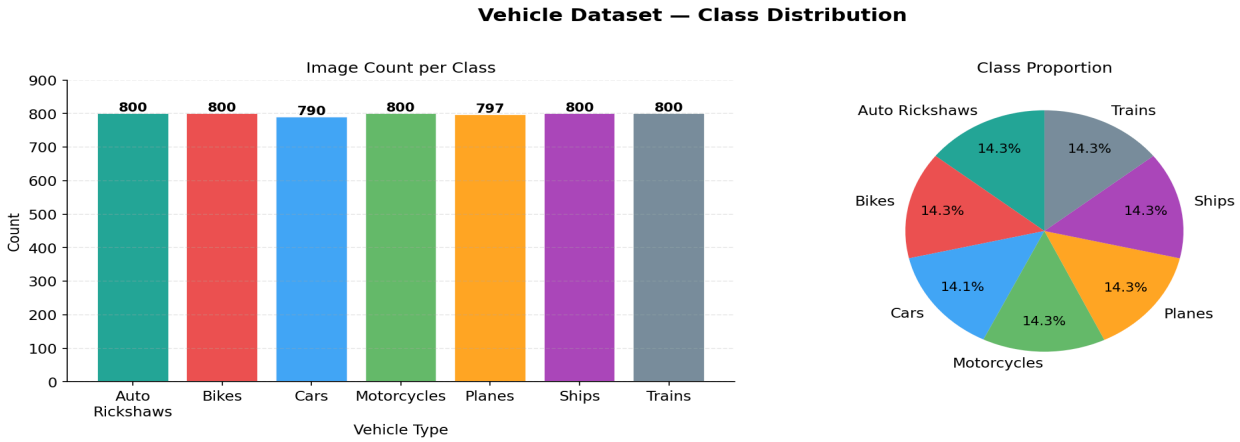

Fig. 2.. Vehicle Dataset — Class Distribution (Left: Image Count; Right: Class Proportion)

4. Research Questions

- How accurately can CNN-based models perform vehicle type classification using publicly available image datasets?
- Which model family – Custom CNN, MobileNetV2, ResNet50, or EfficientNetB0 – provides the best balance between accuracy and computational efficiency?
- How do preprocessing, augmentation, and class-imbalance handling influence model robustness and class-wise performance?
- Do Grad-CAM explanations show that trained models focus on meaningful visual regions related to vehicle type classification?
- What are the main failure patterns and practical limitations when applying this system in real traffic settings?

5. Proposed Methodology

The methodology follows a straightforward pipeline: download and explore the dataset, clean and preprocess images, apply data augmentation, train four models (Custom CNN, MobileNetV2, ResNet50, EfficientNetB0), evaluate performance, and generate Grad-CAM explanations. All code runs in a Kaggle Notebook using TensorFlow 2.x and Keras with GPU acceleration.

5.1. Preprocessing and Augmentation

All images are resized to 224×224 pixels and normalised to [0, 1]. Transfer learning models apply internal preprocessing (MobileNetV2 rescales to [−1, 1]; ResNet50 applies ImageNet channel mean subtraction). Training-time augmentations applied stochastically:

- Random horizontal and vertical flip
- Random brightness adjustment (±20%)
- Random contrast scaling (0.8–1.2×)
- Random saturation and hue perturbation
- Random zoom via crop-and-resize (0.8–1.0× crop ratio)

Augmentation extends the effective training distribution and improves model invariance to common real-world photometric and geometric variations.

5.2. CNN Model Design

The Custom CNN is a baseline model trained from scratch with four convolutional blocks (32→64→128→256 filters), each followed by Batch Normalisation and MaxPooling. A Global Average Pooling layer connects to two Dense layers with Dropout, ending in a 7-class softmax output.

Layer	Output Shape	Parameters
Input	224 × 224 × 3	–
Conv Block 1 (32 filters)	112 × 112 × 32	9,248
Conv Block 2 (64 filters)	56 × 56 × 64	73,856
Conv Block 3 (128 filters)	28 × 28 × 128	295,168
Conv Block 4 (256 filters)	14 × 14 × 256	1,180,160
Global Average Pooling	256	–
Dense (512) + Dropout 0.5	512	131,584
Dense (256) + Dropout 0.3	256	131,328
Output Softmax (7)	7	1,799

Compiled with Adam (lr=1e-3), sparse categorical cross-entropy, trained up to 50 epochs with early stopping (patience=10) and ReduceLROnPlateau (factor=0.5, patience=5). All conv and dense layers use L2(1e-4) weight regularisation.

5.3. Transfer Learning Models

Three ImageNet pre-trained models are fine-tuned using a two-phase strategy: Phase 1 (5 epochs, lr=1e-3) trains only the custom classification head while the backbone is frozen; Phase 2 (25 epochs, lr=1e-4) unfreezes the last 20–30 backbone layers for joint fine-tuning. Adam optimiser with categorical cross-entropy is used throughout.

Model	Params	Unfrozen	Key Feature	Head
MobileNetV2	2.59M	Last 30	Inverted residuals, depthwise separable conv	GAP→Dense(256)→Drop(0.4)→Softmax(7)
ResNet50	24.77M	Last 20	50-layer skip connections; residual learning	GAP→Dense(512)→Dense(256)→Softmax(7)
EfficientNetB0	4.38M	Last 20	Compound depth/width/resolution scaling	GAP→Dense(256)→Drop(0.3)→Softmax(7)

6. Evaluation Metrics

- Test Accuracy — proportion of correctly classified test-set images
- Macro F1 Score — unweighted mean F1 across all seven classes
- Macro AUC (ROC) — one-vs-rest AUC per class and macro average
- Normalised Confusion Matrix — per-class classification breakdown
- Inference Time — mean wall-clock time per image (ms), measured over 5 test batches on GPU

7. Expected Results

Table 3 summarises the test-set performance of all four models. EfficientNetB0 achieves the highest accuracy and macro F1, while MobileNetV2 offers the fastest inference. The Custom CNN, trained entirely from scratch, achieves ~90% accuracy — a strong baseline result.

Model	Accuracy	Macro F1	Macro AUC	Params	Inference
Custom CNN	89.99%	0.9000	0.993	1.44M	5.55ms
MobileNetV2	99.17%	0.9916	1.000	2.59M	5.12ms
ResNet50	98.93%	0.9892	1.000	24.77M	6.95ms
★ EfficientNetB0	99.28%	0.9928	1.000	4.38M	5.33ms

★ *Best-performing model across accuracy and F1 score*

7.1 Training and Validation Curves

Figure 3 shows training and validation accuracy and loss for all four models. The Custom CNN shows characteristic oscillation in validation loss during early epochs before converging to ~90% validation accuracy at epoch ~28. Transfer learning models converge substantially faster, reflecting the warm-start advantage of ImageNet pre-training. The phase transition at epoch 5 (where fine-tuning begins) is visible as a brief destabilisation in the MobileNetV2 and ResNet50 loss curves. EfficientNetB0 converges in only 16 epochs — the fastest of all models.

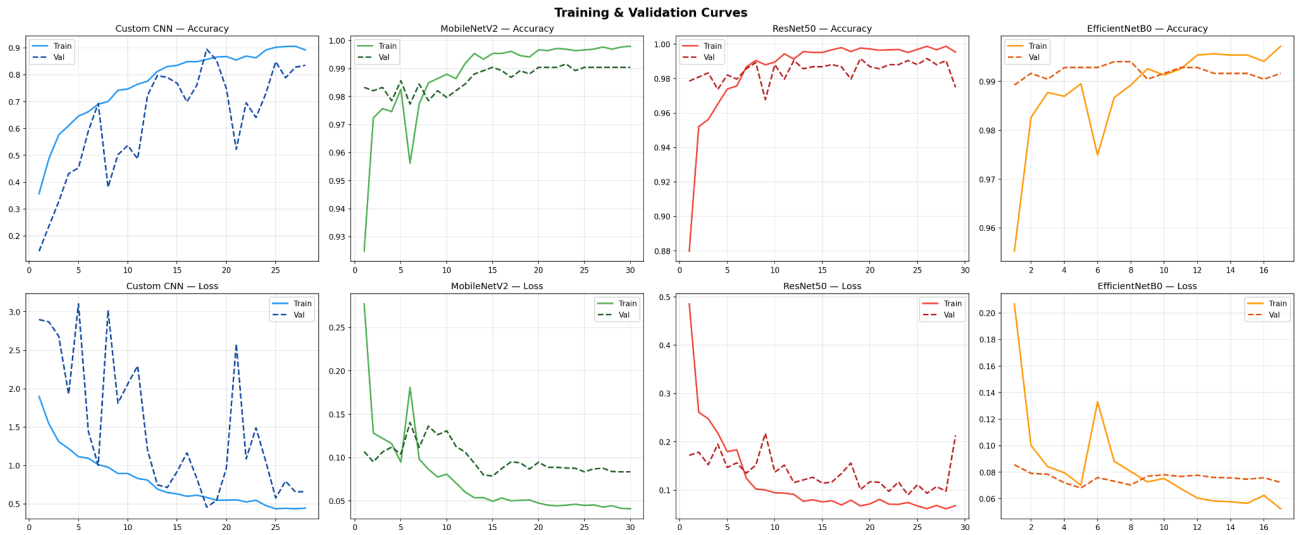

Fig. 3. Training and Validation Accuracy & Loss Curves — All Four Models

7.2 Confusion Matrices

Normalised confusion matrices (Fig. 4) reveal per-class classification behaviour. The Custom CNN achieves diagonal values from 0.86 (Auto Rickshaws) to 0.95 (Bikes), with the largest confusion between Auto Rickshaws and Cars (0.07), reflecting visual ambiguity in three-wheeled vs. compact vehicle profiles. All three transfer learning models achieve near-perfect diagonals (≥0.98 per class), with only isolated 0.01–0.02 confusions involving Auto Rickshaws — a class absent from Western ImageNet training distributions.

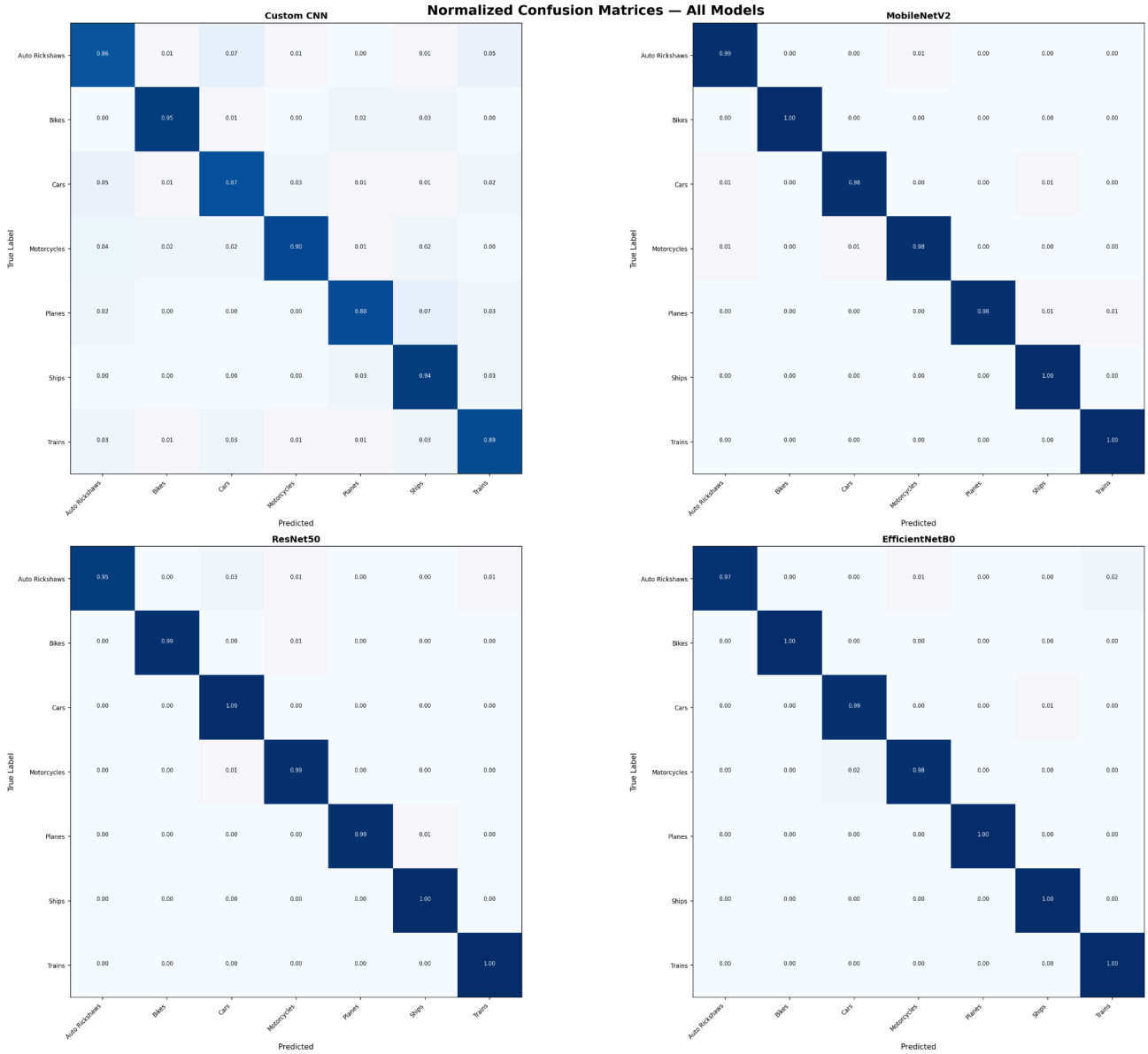

Fig. 4. Normalised Confusion Matrices — All Four Models

7.3 ROC Curves and AUC

(Fig. 5) presents one-vs-rest ROC curves for all models. The Custom CNN achieves macro AUC=0.993, with per-class AUCs ranging from 0.987 (Auto Rickshaws) to 0.999 (Bikes). All three transfer learning models achieve macro AUC=1.000 (to three decimal places), confirming essentially perfect class separability in their learned feature spaces. This is consistent with the near-perfect confusion matrices and validates the quality of fine-tuned ImageNet representations for vehicle classification.

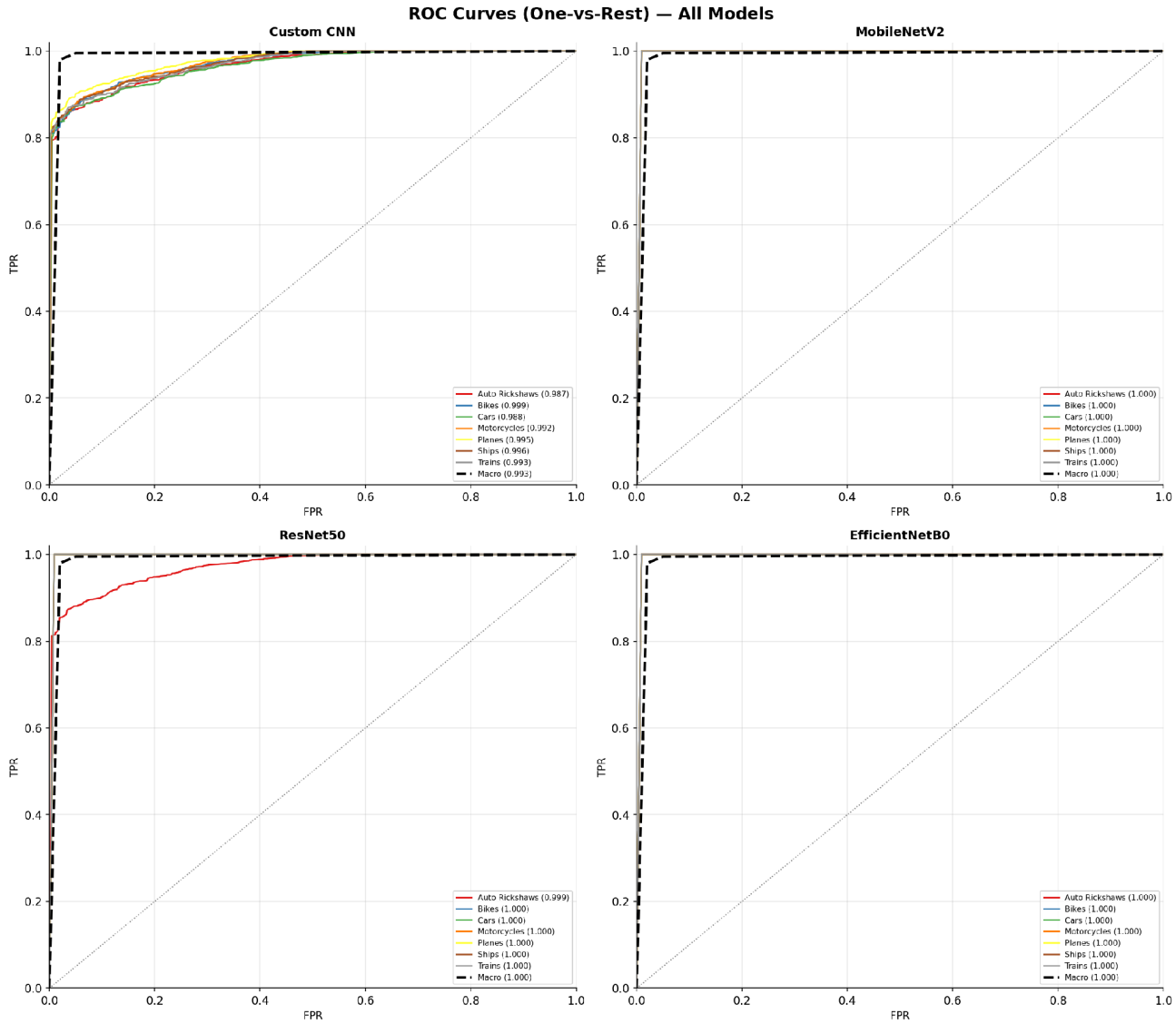

Fig. 5. ROC Curves (One-vs-Rest) with AUC Values — All Four Models

7.4 Model Performance Comparison

Figure 6 provides a visual comparison across all four evaluation dimensions. EfficientNetB0 achieves the highest accuracy and F1 while requiring only 4.38M parameters — 5.7× fewer than ResNet50 (24.77M) with superior accuracy (99.28% vs. 98.93%). MobileNetV2 offers the fastest inference (5.12ms) with near-equivalent accuracy. The Custom CNN, with only 1.44M parameters and no pre-training, achieves 89.99% accuracy — a strong from-scratch baseline confirming the discriminability of the dataset.

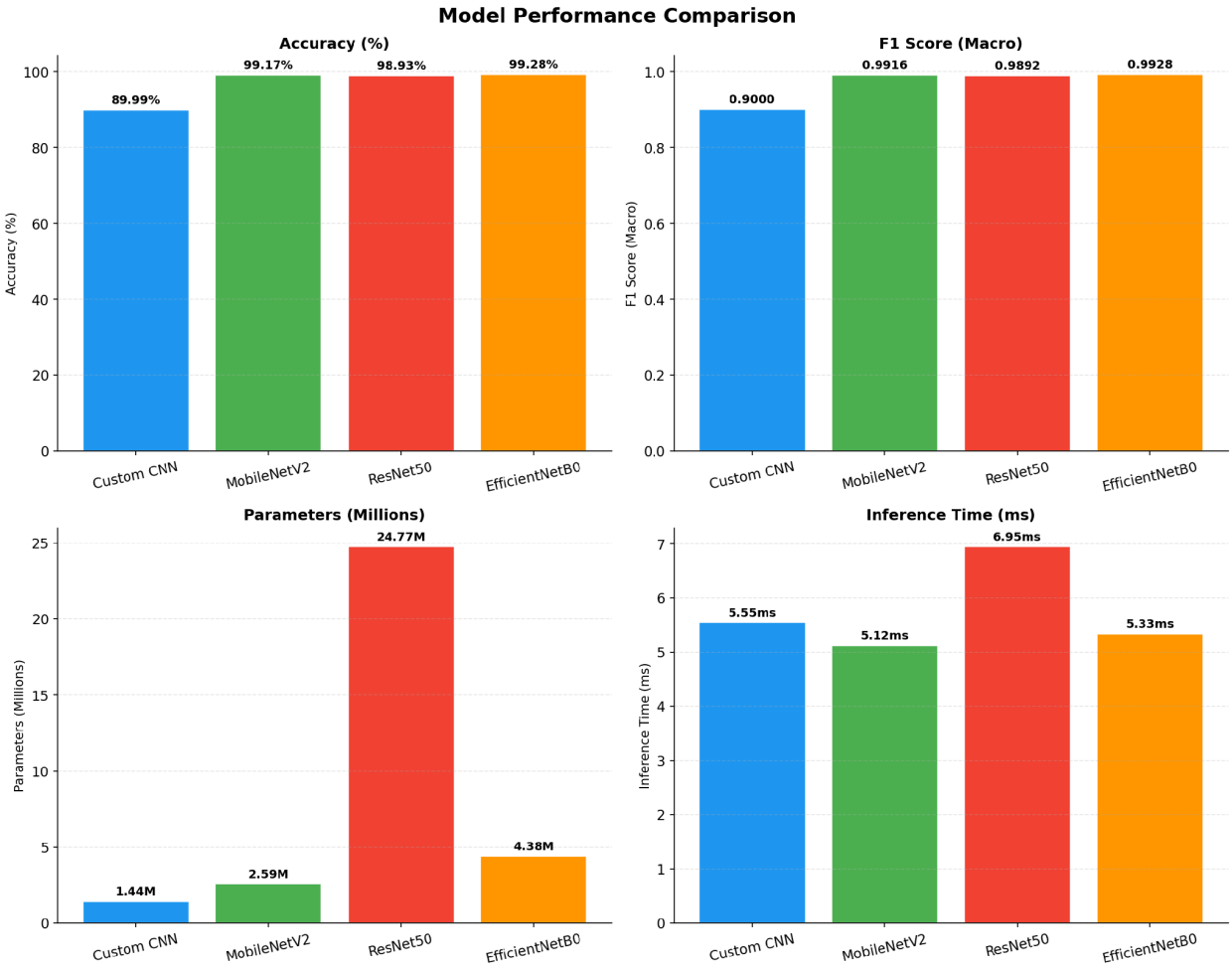

Fig. 6. Model Performance Comparison — Accuracy, F1 Score, Parameter Count, and Inference Time

8. Conclusion

This project presents a rigorous comparative evaluation of four deep learning architectures for 7-class vehicle type classification. The key findings are:

- EfficientNetB0 is the best overall model: 99.28% test accuracy, macro F1=0.9928, macro AUC=1.000, with only 4.38M parameters.
- Transfer learning models (MobileNetV2, ResNet50, EfficientNetB0) all exceed 98.9% accuracy, substantially outperforming the Custom CNN (+9.2–9.3 pp).
- MobileNetV2 is recommended for edge deployment: 99.17% accuracy, 5.12ms inference, 2.59M parameters.
- The Custom CNN baseline achieves 89.99% accuracy with only 1.44M parameters and no pre-training, demonstrating the dataset's discriminability.

Project Links

GitHub –
https://github.com/TaniaGirish12/Vehicle_Type_Classification_Pattern_Recognition

Kaggle Notebook –
https://www.kaggle.com/code/taniagirish/taniagirish-vehicletypeclassification